K L Deemed to be University

(Koneru Lakshmaiah Education Foundation)

GREEN FIELDS, VADDESWARAM · TERM 3, AY 2025-2026 · PROJECT-BASED LEARNING

MID-TERM EXAMINATION · PAPER SET 1

COURSE	DATA STRUCTURES AND ALGORITHMS - 2	**CODE**	25SC1305E
EXAMINATION	Mid-Term Examination	**PAPER**	Set 1
DURATION	90 minutes	**MAX. MARKS**	45
COORDINATOR	Zeelan Basha CMAK	**MODE**	Project-Based Learning

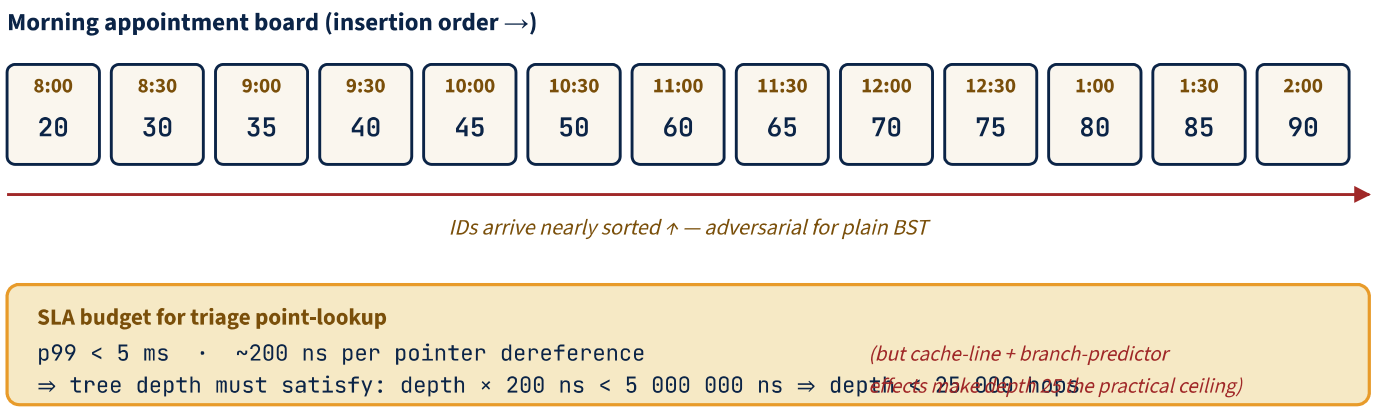
INSTRUCTIONS TO CANDIDATES

- Answer ALL three questions. Each question carries 15 marks. Maximum: 45 marks.
- Each question is a scenario / case-study / micro-project covering one Course Outcome.
- Show all working, intermediate steps, code listings, diagrams, derivations and reasoning traces.
- Time allowed: 90 minutes. Calculator allowed. No mobile phones, smart watches, or AI assistants.

Q1. CO1 | Module 1: BST & AVL Trees [15 Marks]

SCENARIO / CASE STUDY

MediFlow Hospital — appointment-driven patient indexing. The on-call patient dictionary at MediFlow is keyed by 8-digit Patient-ID. The day's IDs are issued in *scheduled appointment order*: the 8 a.m. block first, then 9 a.m., 10 a.m., and so on; within each block the registrar enters appointments in the order they were booked, which is itself near-monotonic by ID because IDs are assigned at booking time. Today's first 13 IDs land in this order: `20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90`. The triage console must answer point-lookups with **p99 < 5 ms**; current hardware does ~200 ns per pointer dereference, so any tree deeper than 25 levels misses the SLA. By noon, three deletions arrive: 30 (transferred), 70 (discharged), and 50 (admission closed) — in that order.



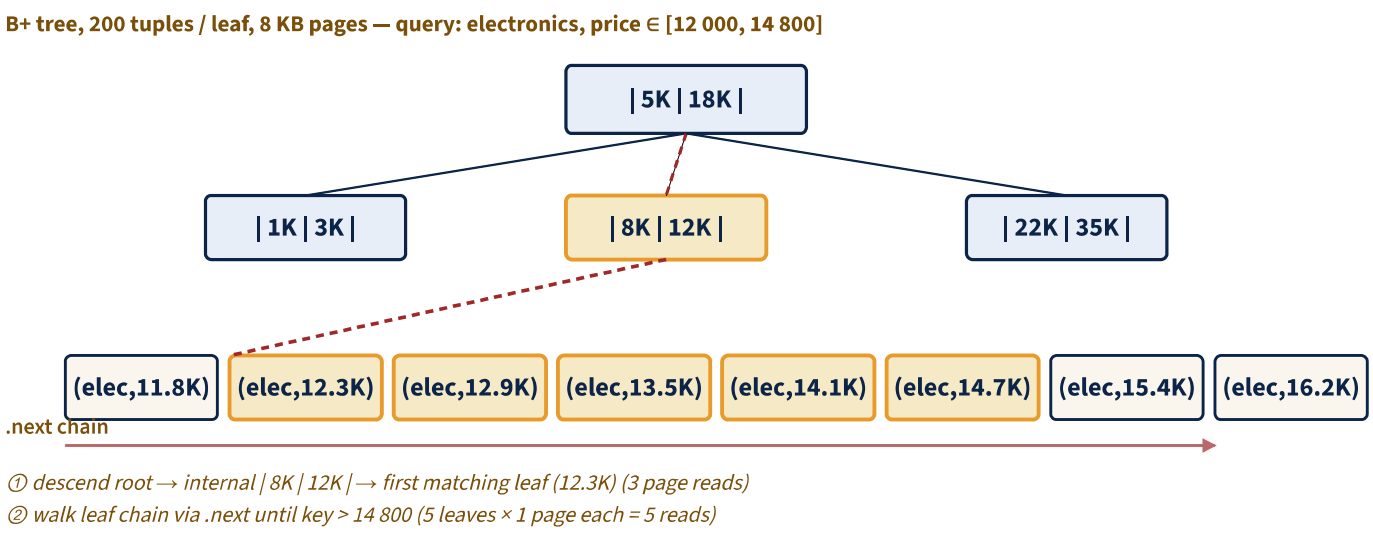
SUB	QUESTION	MARKS	CO	BTL
(a)	Construct the **plain BST** from the appointment-order insertion sequence given in the scenario. State its height (longest root-to-leaf path in edges). Using the SLA budget shown in the diagram (200 ns per hop, 5 ms ceiling), do *one explicit numeric calculation* showing whether a worst-case point-lookup on this BST would meet or violate the p99 SLA. Briefly explain the structural reason this insertion order is adversarial for plain BST.	5	CO1	BTL3
(b)	Now build an **AVL tree** from the same insertion sequence. After every insertion that triggers rebalancing, name the rotation (LL / RR / LR / RL), identify the pivot node, and state the balance factors of the two nodes adjacent to the rotation, immediately after the rotation. Show the final AVL tree. Then, complete the boilerplate Python snippet below so it implements `insert(node, key)` and the two single-rotation helpers consistently with your hand-traced tree. (Boilerplate: `AVLNode`, `get_height`, `get_balance`, `update_height` are already given — fill the three TODOs.)	7	CO1	BTL3
(c)	After the noon deletions (30, 70, 50, in that order) are applied to the AVL tree from (b), show the final tree and any rotations triggered during deletion. Then **analyse**: compute the worst-case lookup time on the post-delete AVL tree and on the post-delete plain-BST equivalent (using the same SLA model from (a)). Quote both as a percentage of the 5 ms budget consumed. State which deletion in the sequence caused the largest structural change and why.	3	CO1	BTL4

```
class AVLNode {
    int key;
    AVLNode left, right;
    int height = 1;
    AVLNode(int key) { this.key = key; }
}

static int  height (AVLNode n) { return n == null ? 0 : n.height; }
static int  balance(AVLNode n) { return n == null ? 0 : height(n.left) - height(n.right); }
static void updateHeight(AVLNode n) {
    if (n != null) n.height = 1 + Math.max(height(n.left), height(n.right));
}

static AVLNode rotateRight(AVLNode y) {
    // TODO 1: perform a right rotation around y; return the new subtree root.
    return null;
}

static AVLNode rotateLeft(AVLNode x) {
    // TODO 2: perform a left rotation around x; return the new subtree root.
    return null;
}

static AVLNode insert(AVLNode node, int key) {
    // TODO 3: standard BST insert + rebalance using the four cases (LL, LR, RL, RR).
    return null;
}
```

Q2. CO2 | Module 2: B–Trees, B+ Trees & Range Query Structures [15 Marks]

SCENARIO / CASE STUDY

Flipkart product catalog — price-range scans dominate. The smartphone catalog has 10^7 products indexed by composite key `(category, price)` in a B+ tree on SSD. Each leaf holds **200** tuples of the form `(category, price, product_id)` packed into an 8 KB page (the disk's natural page size). A production trace over Diwali week shows: 95 % of catalog queries are price-range scans of the form *'all products in `category=X` with `price ∈ [lo, hi]`'*; only 5 % are direct product-page lookups. Of the range queries, 87 % have a range width within ±20 % of the category's average price, returning roughly 2 800 products on average per query. Engineers are debating whether to also maintain a hash index for short ranges.



SUB	QUESTION	MARKS	CO	BTL
(a)	Using the scenario's numbers — 10^7 products, 200 tuples per leaf, 8 KB pages — derive the **tree height** and the **I/O cost** (number of page reads) for the typical range query 'electronics, price ∈ [lo, hi]' that returns ~2 800 products. Assume internal-node fanout is also 200. Show the calculation explicitly: descent cost + leaf-chain walk cost. State which of these two terms dominates and why.	5	CO2	BTL3
(b)	An engineer's draft of `range_count(root, lo, hi)` is shown below. It is intended to return the count of keys in `[lo, hi]`. **Two performance bugs are hiding in it that will hurt p99 on Flipkart's workload** — identify both, name the bug, and quantify the I/O impact for the average query (returning ~2 800 results from a 10^7-row tree). Then write the corrected version (you may overwrite the function body in your answer).	6	CO2	BTL4
(c)	The team proposes adding a **secondary hash index** on `(category, price)` to accelerate *short-range* queries (range width returning < 50 products). Using the trace data — 87 % of range queries fall within ±20 % of the category mean and return ~2 800 products on average — argue quantitatively whether the hash index would be a net win, a net loss, or workload-dependent. Identify the smallest range width at which the hash index would actually beat the B+ tree's leaf-chain walk on this hardware (8 KB pages, 200 tuples/leaf). Be explicit about the constants you assume.	4	CO2	BTL5

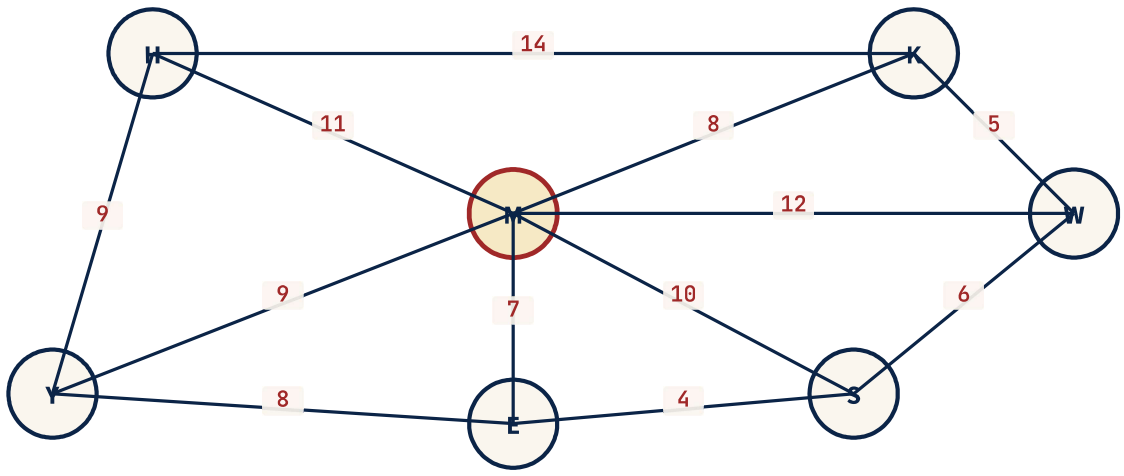
```
class BPlusNode {
    boolean isLeaf;
    int[] keys;
    BPlusNode[] children;   // internal: child pointers
    BPlusNode next;         // leaf-chain pointer (null on internal nodes)
}

static int rangeCount(BPlusNode root, int lo, int hi) {
    BPlusNode leaf = root;
    while (!leaf.isLeaf) {
        leaf = leaf.children[0];        // always go to leftmost child
    }
    int count = 0;
    while (leaf != null) {
        for (int k : leaf.keys) {
            if (lo <= k && k <= hi) count++;
        }
        leaf = leaf.next;               // never break; walk to end
    }
    return count;
}
```

Q3. CO3 | Module 3: Graph Traversal & Minimum Spanning Trees [15 Marks]

SCENARIO / CASE STUDY

Bangalore Metro Phase-3 expansion — minimum-cost connection plan with redundancy. The civil-engineering team must connect **6 candidate new stations** {`K (Kadugodi)`, `W (Whitefield)`, `S (Sarjapur)`, `E (Electronic City Ext)`, `Y (Yeshwanthpur West)`, `H (Hebbal North)`} into the existing network via the central hub `M (Majestic)`. Each candidate edge has a cost in ₹-crore reflecting terrain, land acquisition, and tunnelling difficulty. **Constraints:** (i) total construction cost must be minimised; (ii) the network must remain connected; (iii) **regulatory redundancy mandate**: there must be at least *two edge-disjoint paths* between Majestic (M) and Whitefield (W) so a single track failure does not isolate the IT corridor.

Candidate edges with construction cost (₹-cr) — redundancy mandate: 2 edge-disjoint M↔W paths



Mandate: ≥ 2 edge-disjoint paths between M (Majestic, central hub) and W (Whitefield, IT corridor) — interchange to existing 41-station network

SUB	QUESTION	MARKS	CO	BTL
(a)	Run **Kruskal's algorithm** on the 12-edge candidate graph shown in the diagram (edges + costs taken from the SVG; list them sorted ascending if helpful). Show the union-find evolution as a sequence of (edge considered, accept/reject, parent-pointer state). State the resulting MST as a list of edges and its total cost in ₹-crore.	5	CO3	BTL3
(b)	The naïve Kruskal implementation below uses union-find *without* union-by-rank and *without* path compression. **Analyse** its worst-case time complexity on a graph with n vertices and `m` edges, then plug in the project's numbers (n = 7 if you include Majestic; m = 12 candidate edges). Compare with the optimised version (union-by-rank + path compression). State concretely how many `find()` pointer-hops the naive version performs in the worst case here.	5	CO3	BTL4
(c)	Verify the **regulatory redundancy mandate**: does the MST you found in (a) provide two edge-disjoint paths between Majestic (M) and Whitefield (W)? If yes, list both paths. If no, propose the *minimum-cost augmentation* — a single additional edge from the candidate set that, when added to the MST, satisfies the mandate — and prove minimality by listing the alternatives and their costs.	5	CO3	BTL5

```
class UnionFindNaive {
    int[] parent;
    UnionFindNaive(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {                       // NO path compression
        while (parent[x] != x) x = parent[x];
        return x;
    }
    boolean union(int x, int y) {           // NO union-by-rank
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        parent[rx] = ry;
        return true;
    }
}

static List<int[]> kruskalNaive(int n, int[][] edges) {
    Arrays.sort(edges, Comparator.comparingInt(e -> e[0])); // by weight asc
    UnionFindNaive uf = new UnionFindNaive(n);
    List<int[]> mst = new ArrayList<>();
    for (int[] e : edges) {                             // e = {w, u, v}
        if (uf.union(e[1], e[2])) mst.add(e);
    }
    return mst;
}
```